

Relatorio da Tarefa 20

Data da revisao: 2026-06-18.

Objetivo

A Tarefa 20 pede executar o exercicio `heat.c` do tutorial de programacao de GPUs com OpenMP, explorar diretivas de paralelizacao e movimentacao de dados entre host e dispositivo, testar tamanhos diferentes e perfilar com `nsys`.

Fonte escolhida

Foram encontradas duas copias da tarefa:

- `AULA-01-P-GPU/tarefa-20`
- `Aula-P-GPU/tarefa-20`

As copias contem os mesmos fontes (`heat.c`, `heat_target.c` e `comparar_resultados.py`) e os mesmos scripts SLURM. As diferencas encontradas estao nos caminhos escritos nos READMEs/scripts. Para evitar destruir trabalho existente, nenhuma copia foi apagada. Este relatorio usa `AULA-01-P-GPU/tarefa-20` como fonte de referencia, porque seus scripts e README ja apontam para essa propria arvore.

O que esta implementado

- `heat.c`: versao CPU original do stencil 2D de transferencia de calor, com solucao manufaturada e erro L2.
- `heat_target.c`: versao OpenMP target com `#pragma omp target map(tofrom: u[0:n*n], u_tmp[0:n*n])` e `#pragma omp loop collapse(2)` no kernel `solve`.
- `run_npad.sbatch`: compila CPU e GPU com NVHPC, executa casos `N:nsteps`, repete medicoes e grava CSV em `resultados/`.
- `run_nsys_npad.sbatch`: compila e perfila CPU, GPU ou ambas com Nsight Systems.
- `comparar_resultados.py`: agrega CSVs, calcula medias, speedups e diferenca de erro L2; tambem gera graficos se `matplotlib` estiver disponivel.

Esta versao GPU corresponde ao exercicio inicial de `heat_target.c` do material: ela offloada o laço do stencil e usa `map` no proprio `target`. Como o `map` esta dentro de `solve()`, chamadas sucessivas podem transferir `u` e `u_tmp` a cada passo de tempo. Esse custo deve ser considerado na comparacao e no perfil.

Validacao local realizada

O ambiente local tinha `gcc`, mas nao tinha `nvc` nem `nsys`. Portanto, nao foi possivel validar offload real nem gerar perfil local. Tambem nao foi tentado NPAD.

Comandos executados localmente:

```
gcc --version
nvc --version
nsys --version
mkdir -p AULA-01-P-GPU/tarefa-20/bin
gcc -O2 -fopenmp AULA-01-P-GPU/tarefa-20/heat.c -o AULA-01-P-GPU/tarefa-20/bin/heat_cpu_gcc -lm
gcc -O2 -fopenmp AULA-01-P-GPU/tarefa-20/heat_target.c -o AULA-01-P-GPU/tarefa-20/bin/heat_target_gcc -lm
AULA-01-P-GPU/tarefa-20/bin/heat_cpu_gcc 100 10
OMP_TARGET_OFFLOAD=DISABLED AULA-01-P-GPU/tarefa-20/bin/heat_target_gcc 100 10
```

Resultados locais relevantes:

Variante	Ambiente	N	Passos	Erro L2	Observacao
heat.c	GCC/OpenMP CPU	100	10	4.015950E-09	Executou localmente
heat_target.c	GCC/OpenMP com offload desabilitado	100	10	4.015950E-09	Executou no host, sem GPU

Essa validacao confirma compilacao local e consistencia numerica basica, mas nao mede desempenho GPU.

Como rodar a comparacao CPU/GPU em ambiente com GPU

Na raiz do repositorio, em um ambiente com SLURM, GPU NVIDIA, NVHPC e permissao para executar jobs:

```
sbatch AULA-01-P-GPU/tarefa-20/run_npad.sbatch
```

Configuracao padrao do script:

```
CASES="1000:10 2000:20 4000:40"
REPEATS=3
OMP_TARGET_OFFLOAD=MANDATORY
```

Para alterar tamanhos e repeticoes:

```
CASES="500:10 1000:10 2000:20" REPEATS=5 sbatch AULA-01-P-GPU/tarefa-20/run_npad.sbatch
```

Depois do job:

```
cd AULA-01-P-GPU/tarefa-20
python3 comparar_resultados.py
```

Arquivos esperados:

- resultados/heat_resultados_<job_id>.csv
- resultados/comparacao_heat_resumo.csv
- resultados/comparacao_heat.md
- resultados/tempo_solve_cpu_gpu.png, se houver `matplotlib`
- resultados/speedup_solve_cpu_gpu.png, se houver `matplotlib`

Critérios de comparacao

Comparar CPU e GPU sempre com o mesmo `N`, mesmo `nsteps`, mesmo compilador base quando possível, mesma particao/no e repeticoes suficientes para reduzir ruido.

Métricas principais:

- `Error (L2norm)`: deve ficar muito proximo entre CPU e GPU; diferencas grandes indicam erro de implementacao ou problema de offload.
- `Solve time (s)`: tempo do kernel iterativo; e a metrica principal de speedup.
- `Total time (s)`: inclui alocao, inicializacao, verificacao e overheads.
- `speedup_solve = tempo medio CPU / tempo medio GPU`.
- `speedup_total = tempo total medio CPU / tempo total medio GPU`.
- `Delta L2 = |erro medio CPU - erro medio GPU|`.

Para esta implementacao, o tempo GPU deve ser interpretado como tempo de computacao mais custo de movimentacao de dados por passo, porque `map(tofrom)` esta dentro de `solve()`.

Perfil com nsys

Para perfilar a versao GPU:

```
sbatch AULA-01-P-GPU/tarefa-20/run_nsys_npad.sbatch
```

Para mudar tamanho:

```
N=2000 NSTEPS=20 sbatch AULA-01-P-GPU/tarefa-20/run_nsys_npad.sbatch
```

Para perfilar variantes:

```
PROFILE_VARIANT=gpu sbatch AULA-01-P-GPU/tarefa-20/run_nsys_npad.sbatch
PROFILE_VARIANT=cpu sbatch AULA-01-P-GPU/tarefa-20/run_nsys_npad.sbatch
```

```
PROFILE_VARIANT=both sbatch AULA-01-P-GPU/tarefa-20/run_nsys_npad.sbatch
```

No resumo do **nsys**, procurar:

- **CUDA Kernel Statistics**: evidencia de kernels lançados por OpenMP offload.
- **CUDA Memory Operation Statistics**: custo de cópias host/dispositivo.
- tempo relativo de kernels versus cópias: se cópias dominarem, isso confirma que a versão com **map(tofrom)** dentro do passo de tempo ainda não controla residência de dados na GPU.

Dependências pendentes de GPU

Pendente por depender de GPU/NVHPC/nsys:

- medir CPU x GPU com **OMP_TARGET_OFFLOAD=MANDATORY**;
- gerar CSVs reais em **resultados/heat_resultados_<job_id>.csv**;
- gerar **comparacao_heat.md** e gráficos a partir de medições reais;
- gerar **.nsys-rep/.sqlite** e resumo **nsys stats**;
- confirmar no perfil se o offload criou kernels e qual foi o peso das cópias.

Próxima melhoria técnica

Depois da medição inicial, a melhoria natural é criar uma segunda versão GPU com dados residentes no dispositivo, usando **target data** ou **target enter data/target exit data** ao redor do laço de tempo. Essa variante deve reduzir transferências repetidas e permitir comparar:

- CPU original;
- GPU com **map(tofrom)** por chamada de **solve**;
- GPU com dados mantidos no dispositivo entre passos.

Códigos fonte C usados nos testes

AULA-01-P-GPU/tarefa-20/heat.c

```
/*
** PROGRAM: heat equation solve
**
** PURPOSE: This program will explore use of an explicit
**          finite difference method to solve the heat
**          equation under a method of manufactured solution (MMS)
**          scheme. The solution has been set to be a simple
**          function based on exponentials and trig functions.
**
**          A finite difference scheme is used on a 1000x1000 cube.
**          A total of 0.5 units of time are simulated.
**
**          The MMS solution has been adapted from
**          G.W. Recktenwald (2011). Finite difference approximations
**          to the Heat Equation. Portland State University.
**
```

```

**
**  USAGE:   Run with two arguments:
**           First is the number of cells.
**           Second is the number of timesteps.
**
**           For example, with 100x100 cells and 10 steps:
**
**           ./heat 100 10
**
**
**  HISTORY: Written by Tom Deakin, Oct 2018
**
*/

#include <stdlib.h>
#include <stdio.h>
#include <math.h>

#include <omp.h>

// Key constants used in this program
#define PI acos(-1.0) // Pi
#define LINE "-----\n" // A line for fancy output

// Function definitions
void initial_value(const int n, const double dx, const double length,
double * restrict u);
void zero(const int n, double * restrict u);
void solve(const int n, const double alpha, const double dx, const double
dt, const double * restrict u, double * restrict u_tmp);
double solution(const double t, const double x, const double y, const
double alpha, const double length);
double l2norm(const int n, const double * restrict u, const int nsteps,
const double dt, const double alpha, const double dx, const double length);

// Main function
int main(int argc, char *argv[]) {

    // Start the total program runtime timer
    double start = omp_get_wtime();

    // Problem size, forms an nxn grid
    int n = 1000;

    // Number of timesteps
    int nsteps = 10;

    // Check for the correct number of arguments
    // Print usage and exits if not correct
    if (argc == 3) {

        // Set problem size from first argument
        n = atoi(argv[1]);

```

```
    if (n < 0) {
        fprintf(stderr, "Error: n must be positive\n");
        exit(EXIT_FAILURE);
    }

    // Set number of timesteps from second argument
    nsteps = atoi(argv[2]);
    if (nsteps < 0) {
        fprintf(stderr, "Error: nsteps must be positive\n");
        exit(EXIT_FAILURE);
    }
}

//
// Set problem definition
//
double alpha = 0.1;           // heat equation coefficient
double length = 1000.0;       // physical size of domain: length x length
square
double dx = length / (n+1);   // physical size of each cell (+1 as don't
simulate boundaries as they are given)
double dt = 0.5 / nsteps;     // time interval (total time of 0.5s)

// Stability requires that  $dt/(dx^2) \leq 0.5$ ,
double r = alpha * dt / (dx * dx);

// Print message detailing runtime configuration
printf("\n");
printf(" MMS heat equation\n\n");
printf(LINE);
printf("Problem input\n\n");
printf(" Grid size: %d x %d\n", n, n);
printf(" Cell width: %E\n", dx);
printf(" Grid length: %lf x %lf\n", length, length);
printf("\n");
printf(" Alpha: %E\n", alpha);
printf("\n");
printf(" Steps: %d\n", nsteps);
printf(" Total time: %E\n", dt*(double)nsteps);
printf(" Time step: %E\n", dt);
printf(LINE);

// Stability check
printf("Stability\n\n");
printf(" r value: %lf\n", r);
if (r > 0.5)
    printf(" Warning: unstable\n");
printf(LINE);

// Allocate two nxn grids
double *u      = malloc(sizeof(double)*n*n);
```

```

double *u_tmp = malloc(sizeof(double)*n*n);
double *tmp;

// Set the initial value of the grid under the MMS scheme
initial_value(n, dx, length, u);
zero(n, u_tmp);

//
// Run through timesteps under the explicit scheme
//

// Start the solve timer
double tic = omp_get_wtime();
for (int t = 0; t < nsteps; ++t) {

    // Call the solve kernel
    // Computes u_tmp at the next timestep
    // given the value of u at the current timestep
    solve(n, alpha, dx, dt, u, u_tmp);

    // Pointer swap
    tmp = u;
    u = u_tmp;
    u_tmp = tmp;
}
// Stop solve timer
double toc = omp_get_wtime();

//
// Check the L2-norm of the computed solution
// against the *known* solution from the MMS scheme
//
double norm = l2norm(n, u, nsteps, dt, alpha, dx, length);

// Stop total timer
double stop = omp_get_wtime();

// Print results
printf("Results\n\n");
printf("Error (L2norm): %E\n", norm);
printf("Solve time (s): %lf\n", toc-tic);
printf("Total time (s): %lf\n", stop-start);
printf(LINE);

// Free the memory
free(u);
free(u_tmp);
}

// Sets the mesh to an initial value, determined by the MMS scheme
void initial_value(const int n, const double dx, const double length,
double * restrict u) {

```

```

double y = dx;
for (int j = 0; j < n; ++j) {
    double x = dx; // Physical x position
    for (int i = 0; i < n; ++i) {
        u[i+j*n] = sin(PI * x / length) * sin(PI * y / length);
        x += dx;
    }
    y += dx; // Physical y position
}

}

// Zero the array u
void zero(const int n, double * restrict u) {

    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            u[i+j*n] = 0.0;
        }
    }

}

// Compute the next timestep, given the current timestep
void solve(const int n, const double alpha, const double dx, const double
dt, const double * restrict u, double * restrict u_tmp) {

    // Finite difference constant multiplier
    const double r = alpha * dt / (dx * dx);
    const double r2 = 1.0 - 4.0*r;

    // Loop over the nxn grid
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {

            // Update the 5-point stencil, using boundary conditions on the edges
            of the domain.
            // Boundaries are zero because the MMS solution is zero there.
            u_tmp[i+j*n] = r2 * u[i+j*n] +
            r * ((i < n-1) ? u[i+1+j*n] : 0.0) +
            r * ((i > 0) ? u[i-1+j*n] : 0.0) +
            r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
            r * ((j > 0) ? u[i+(j-1)*n] : 0.0);
        }
    }

}

// True answer given by the manufactured solution
double solution(const double t, const double x, const double y, const
double alpha, const double length) {

```

```

    return exp(-2.0*alpha*PI*PI*t/(length*length)) * sin(PI*x/length) *
sin(PI*y/length);
}

// Computes the L2-norm of the computed grid and the MMS known solution
// The known solution is the same as the boundary function.
double l2norm(const int n, const double * restrict u, const int nsteps,
const double dt, const double alpha, const double dx, const double length)
{
    // Final (real) time simulated
    double time = dt * (double)nsteps;

    // L2-norm error
    double l2norm = 0.0;

    // Loop over the grid and compute difference of computed and known
    solutions as an L2-norm
    double y = dx;
    for (int j = 0; j < n; ++j) {
        double x = dx;
        for (int i = 0; i < n; ++i) {
            double answer = solution(time, x, y, alpha, length);
            l2norm += (u[i+j*n] - answer) * (u[i+j*n] - answer);

            x += dx;
        }
        y += dx;
    }

    return sqrt(l2norm);
}

```

AULA-01-P-GPU/tarefa-20/heat_target.c

```

/*
** PROGRAM: heat equation solve
**
** PURPOSE: This program will explore use of an explicit
**          finite difference method to solve the heat
**          equation under a method of manufactured solution (MMS)
**          scheme. The solution has been set to be a simple
**          function based on exponentials and trig functions.
**
**          A finite difference scheme is used on a 1000x1000 cube.
**          A total of 0.5 units of time are simulated.

```

```

**
**      The MMS solution has been adapted from
**      G.W. Recktenwald (2011). Finite difference approximations
**      to the Heat Equation. Portland State University.
**
**
**  USAGE:  Run with two arguments:
**           First is the number of cells.
**           Second is the number of timesteps.
**
**           For example, with 100x100 cells and 10 steps:
**
**           ./heat 100 10
**
**
**  HISTORY: Written by Tom Deakin, Oct 2018
**
*/

#include <stdlib.h>
#include <stdio.h>
#include <math.h>

#include <omp.h>

// Key constants used in this program
#define PI acos(-1.0) // Pi
#define LINE "-----\n" // A line for fancy output

// Function definitions
void initial_value(const int n, const double dx, const double length,
double * restrict u);
void zero(const int n, double * restrict u);
void solve(const int n, const double alpha, const double dx, const double
dt, const double * restrict u, double * restrict u_tmp);
double solution(const double t, const double x, const double y, const
double alpha, const double length);
double l2norm(const int n, const double * restrict u, const int nsteps,
const double dt, const double alpha, const double dx, const double length);

// Main function
int main(int argc, char *argv[]) {

    // Start the total program runtime timer
    double start = omp_get_wtime();

    // Problem size, forms an nxn grid
    int n = 1000;

    // Number of timesteps
    int nsteps = 10;

    // Check for the correct number of arguments

```

```
// Print usage and exits if not correct
if (argc == 3) {

    // Set problem size from first argument
    n = atoi(argv[1]);
    if (n < 0) {
        fprintf(stderr, "Error: n must be positive\n");
        exit(EXIT_FAILURE);
    }

    // Set number of timesteps from second argument
    nsteps = atoi(argv[2]);
    if (nsteps < 0) {
        fprintf(stderr, "Error: nsteps must be positive\n");
        exit(EXIT_FAILURE);
    }
}

//
// Set problem definition
//
double alpha = 0.1;           // heat equation coefficient
double length = 1000.0;       // physical size of domain: length x length
square
double dx = length / (n+1);   // physical size of each cell (+1 as don't
simulate boundaries as they are given)
double dt = 0.5 / nsteps;     // time interval (total time of 0.5s)

// Stability requires that  $dt/(dx^2) \leq 0.5$ ,
double r = alpha * dt / (dx * dx);

// Print message detailing runtime configuration
printf("\n");
printf(" MMS heat equation\n\n");
printf(LINE);
printf("Problem input\n\n");
printf(" Grid size: %d x %d\n", n, n);
printf(" Cell width: %E\n", dx);
printf(" Grid length: %lf x %lf\n", length, length);
printf("\n");
printf(" Alpha: %E\n", alpha);
printf("\n");
printf(" Steps: %d\n", nsteps);
printf(" Total time: %E\n", dt*(double)nsteps);
printf(" Time step: %E\n", dt);
printf(LINE);

// Stability check
printf("Stability\n\n");
printf(" r value: %lf\n", r);
if (r > 0.5)
    printf(" Warning: unstable\n");
```

```
printf(LINE);

// Allocate two nxn grids
double *u      = malloc(sizeof(double)*n*n);
double *u_tmp = malloc(sizeof(double)*n*n);
double *tmp;

// Set the initial value of the grid under the MMS scheme
initial_value(n, dx, length, u);
zero(n, u_tmp);

//
// Run through timesteps under the explicit scheme
//

// Start the solve timer
double tic = omp_get_wtime();
for (int t = 0; t < nsteps; ++t) {

    // Call the solve kernel
    // Computes u_tmp at the next timestep
    // given the value of u at the current timestep
    solve(n, alpha, dx, dt, u, u_tmp);

    // Pointer swap
    tmp = u;
    u = u_tmp;
    u_tmp = tmp;
}
// Stop solve timer
double toc = omp_get_wtime();

//
// Check the L2-norm of the computed solution
// against the *known* solution from the MMS scheme
//
double norm = l2norm(n, u, nsteps, dt, alpha, dx, length);

// Stop total timer
double stop = omp_get_wtime();

// Print results
printf("Results\n\n");
printf("Error (L2norm): %E\n", norm);
printf("Solve time (s): %lf\n", toc-tic);
printf("Total time (s): %lf\n", stop-start);
printf(LINE);

// Free the memory
free(u);
free(u_tmp);

}
```

```

// Sets the mesh to an initial value, determined by the MMS scheme
void initial_value(const int n, const double dx, const double length,
double * restrict u) {

    double y = dx;
    for (int j = 0; j < n; ++j) {
        double x = dx; // Physical x position
        for (int i = 0; i < n; ++i) {
            u[i+j*n] = sin(PI * x / length) * sin(PI * y / length);
            x += dx;
        }
        y += dx; // Physical y position
    }
}

// Zero the array u
void zero(const int n, double * restrict u) {
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {
            u[i+j*n] = 0.0;
        }
    }
}

// Compute the next timestep, given the current timestep
void solve(const int n, const double alpha, const double dx, const double
dt, const double * restrict u, double * restrict u_tmp) {

    // Finite difference constant multiplier
    const double r = alpha * dt / (dx * dx);
    const double r2 = 1.0 - 4.0*r;

    // Loop over the nxn grid
    #pragma omp target map(tofrom: u[0:n*n], u_tmp[0:n*n])
    #pragma omp loop collapse(2)
    for (int i = 0; i < n; ++i) {
        for (int j = 0; j < n; ++j) {

            // Update the 5-point stencil, using boundary conditions on the edges
            of the domain.
            // Boundaries are zero because the MMS solution is zero there.
            u_tmp[i+j*n] = r2 * u[i+j*n] +
            r * ((i < n-1) ? u[i+1+j*n] : 0.0) +
            r * ((i > 0) ? u[i-1+j*n] : 0.0) +
            r * ((j < n-1) ? u[i+(j+1)*n] : 0.0) +
            r * ((j > 0) ? u[i+(j-1)*n] : 0.0);
        }
    }
}

```

```

// True answer given by the manufactured solution
double solution(const double t, const double x, const double y, const
double alpha, const double length) {

    return exp(-2.0*alpha*PI*PI*t/(length*length)) * sin(PI*x/length) *
sin(PI*y/length);

}

// Computes the L2-norm of the computed grid and the MMS known solution
// The known solution is the same as the boundary function.
double l2norm(const int n, const double * restrict u, const int nsteps,
const double dt, const double alpha, const double dx, const double length)
{

    // Final (real) time simulated
    double time = dt * (double)nsteps;

    // L2-norm error
    double l2norm = 0.0;

    // Loop over the grid and compute difference of computed and known
solutions as an L2-norm
    double y = dx;
    for (int j = 0; j < n; ++j) {
        double x = dx;
        for (int i = 0; i < n; ++i) {
            double answer = solution(time, x, y, alpha, length);
            l2norm += (u[i+j*n] - answer) * (u[i+j*n] - answer);

            x += dx;
        }
        y += dx;
    }

    return sqrt(l2norm);

}

```

Scripts sbatch do NPAD

AULA-01-P-GPU/tarefa-20/run_npad.sbatch

```

#!/bin/bash
#SBATCH --partition=gpu-4-a100
#SBATCH --gpus-per-node=1
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8

```

```

#SBATCH --time=00:30:00
#SBATCH --job-name=t20-heat
#SBATCH --output=tarefa20-%j.slurm.out
#SBATCH --error=tarefa20-%j.slurm.err

set -euo pipefail

ROOT="${SLURM_SUBMIT_DIR:-$(pwd)}"
if [[ -d "${ROOT}/AULA-01-P-GPU/tarefa-20" ]]; then
    cd "${ROOT}/AULA-01-P-GPU/tarefa-20"
else
    cd "$(dirname "$0")"
fi

mkdir -p bin resultados

JOB_ID="${SLURM_JOB_ID:-local}"
exec > >(tee -a "resultados/tarefa20-${JOB_ID}.out")
exec 2> >(tee -a "resultados/tarefa20-${JOB_ID}.err" >&2)

module load compilers/nvidia/nvhpc/24.11

NVC="${NVC:-nvc}"
CPU_FLAGS="${CPU_FLAGS:--fast -mp=multicore}"
GPU_FLAGS="${GPU_FLAGS:--fast -mp=gpu}"

echo "Compilador:"
"${NVC}" --version
echo

echo "Compilando versao CPU..."
"${NVC}" ${CPU_FLAGS} heat.c -o bin/heat_cpu -lm

echo "Compilando versao GPU..."
"${NVC}" ${GPU_FLAGS} heat_target.c -o bin/heat_gpu -lm

CSV="resultados/heat_resultados_${JOB_ID}.csv"
CASES="${CASES:-1000:10 2000:20 4000:40}"
REPEATS="${REPEATS:-3}"
export OMP_NUM_THREADS="${OMP_NUM_THREADS:-${SLURM_CPUS_PER_TASK:-1}}"
export OMP_TARGET_OFFLOAD="${OMP_TARGET_OFFLOAD:-MANDATORY}"

printf
"job_id,variant,executable,n,nsteps,repeat,status,error_l2,solve_time_s,tot
al_time_s,omp_threads,omp_devices,offload_policy\n" > "${CSV}"

run_case() {
    local variant="$1"
    local executable="$2"
    local n="$3"
    local nsteps="$4"
    local repeat="$5"
    local
    raw="resultados/raw_${variant}_n${n}_steps${nsteps}_r${repeat}_${JOB_ID}.lo

```

```

g"

    echo "Executando ${variant}: n=${n}, nsteps=${nsteps},
repeat=${repeat}"

    set +e
    output="$("${executable}" "${n}" "${nsteps}" 2>&1)"
    status="$?"
    set -e

    printf "%s\n" "${output}" > "${raw}"

    error_l2="$("${printf "%s\n" "${output}" | awk -F: '/Error \\\(L2norm\\\\)/
{gsub(/\\[ \\t]/, "", $2); print $2; exit}')"
    solve_time="$("${printf "%s\n" "${output}" | awk -F: '/Solve time \\\(s\\\\)/
{gsub(/\\[ \\t]/, "", $2); print $2; exit}')"
    total_time="$("${printf "%s\n" "${output}" | awk -F: '/Total time \\\(s\\\\)/
{gsub(/\\[ \\t]/, "", $2); print $2; exit}')"

    printf "%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s,%s\n" \
        "${JOB_ID}" \
        "${variant}" \
        "${executable}" \
        "${n}" \
        "${nsteps}" \
        "${repeat}" \
        "${status}" \
        "${error_l2}" \
        "${solve_time}" \
        "${total_time}" \
        "${OMP_NUM_THREADS}" \
        "${OMP_NUM_DEVICES:-unknown}" \
        "${OMP_TARGET_OFFLOAD}" >> "${CSV}"

    if [[ "${status}" -ne 0 ]]; then
        echo "Falha em ${variant} n=${n} nsteps=${nsteps}. Veja ${raw}."
    fi
}

echo "Configuracao:"
echo "  CASES=${CASES}"
echo "  REPEATS=${REPEATS}"
echo "  OMP_NUM_THREADS=${OMP_NUM_THREADS}"
echo "  OMP_TARGET_OFFLOAD=${OMP_TARGET_OFFLOAD}"
echo "  CSV=${CSV}"
echo

for case_spec in ${CASES}; do
    IFS=: read -r n nsteps <<< "${case_spec}"
    for repeat in $(seq 1 "${REPEATS}"); do
        run_case "cpu" "./bin/heat_cpu" "${n}" "${nsteps}" "${repeat}"
        run_case "gpu" "./bin/heat_gpu" "${n}" "${nsteps}" "${repeat}"
    done
done

```

```

done
done

echo
echo "Resultados gravados em ${CSV}"
echo "Para gerar a comparacao:"
echo "  cd AULA-01-P-GPU/tarefa-20"
echo "  python3 comparar_resultados.py"

```

AULA-01-P-GPU/tarefa-20/run_nsys_npad.sbatch

```

#!/bin/bash
#SBATCH --partition=gpu-4-a100
#SBATCH --gpus-per-node=1
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --cpus-per-task=8
#SBATCH --time=00:20:00
#SBATCH --job-name=t20-heat-nsys
#SBATCH --output=nsys-%j.slurm.out
#SBATCH --error=nsys-%j.slurm.err

set -euo pipefail

ROOT="${SLURM_SUBMIT_DIR:-$(pwd)}"
if [[ -d "${ROOT}/AULA-01-P-GPU/tarefa-20" ]]; then
    cd "${ROOT}/AULA-01-P-GPU/tarefa-20"
else
    cd "$(dirname "$0")"
fi

mkdir -p bin resultados/perfis

JOB_ID="${SLURM_JOB_ID:-local}"
exec > >(tee -a "resultados/nsys-${JOB_ID}.out")
exec 2> >(tee -a "resultados/nsys-${JOB_ID}.err" >&2)

module load compilers/nvidia/nvhpc/24.11

if ! command -v nsys >/dev/null 2>&1; then
    module load nsight-systems 2>/dev/null || true
fi

if ! command -v nsys >/dev/null 2>&1; then
    echo "Erro: comando nsys nao encontrado. Verifique os modulos disponiveis com: module avail nsight" >&2
    exit 1
fi

NVC="${NVC:-nvc}"
CPU_FLAGS="${CPU_FLAGS:--fast -mp=multicore}"

```

```

GPU_FLAGS="${GPU_FLAGS:--fast -mp=gpu}"
N="${N:-1000}"
NSTEPS="${NSTEPS:-10}"
PROFILE_VARIANT="${PROFILE_VARIANT:-gpu}"
export OMP_NUM_THREADS="${OMP_NUM_THREADS:-${SLURM_CPUS_PER_TASK:-1}}"
export OMP_TARGET_OFFLOAD="${OMP_TARGET_OFFLOAD:-MANDATORY}"

echo "Configuracao do perfil:"
echo "  PROFILE_VARIANT=${PROFILE_VARIANT}"
echo "  N=${N}"
echo "  NSTEPS=${NSTEPS}"
echo "  OMP_NUM_THREADS=${OMP_NUM_THREADS}"
echo "  OMP_TARGET_OFFLOAD=${OMP_TARGET_OFFLOAD}"
echo

echo "Compilador:"
"${NVC}" --version
echo

echo "Nsight Systems:"
nsys --version
echo

profile_gpu() {
    echo "Compilando GPU..."
    "${NVC}" ${GPU_FLAGS} heat_target.c -o bin/heat_gpu -lm

    local
    output="resultados/perfis/nsys_heat_gpu_n${N}_s${NSTEPS}_${JOB_ID}"
    echo "Gerando perfil GPU em ${output}.nsys-rep"

    nsys profile \
        --trace=cuda,nvtx,osrt \
        --stats=true \
        --force-overwrite=true \
        -o "${output}" \
        ./bin/heat_gpu "${N}" "${NSTEPS}"

    echo
    echo "Resumo GPU:"
    nsys stats "${output}.nsys-rep"
}

profile_cpu() {
    echo "Compilando CPU..."
    "${NVC}" ${CPU_FLAGS} heat.c -o bin/heat_cpu -lm

    local
    output="resultados/perfis/nsys_heat_cpu_n${N}_s${NSTEPS}_${JOB_ID}"
    echo "Gerando perfil CPU em ${output}.nsys-rep"

    nsys profile \
        --trace=osrt,nvtx \
        --stats=true \

```

```
--force-overwrite=true \  
-o "${output}" \  
./bin/heat_cpu "${N}" "${NSTEPS}"  
  
echo  
echo "Resumo CPU:"  
nsys stats "${output}.nsys-rep"  
}  
  
case "${PROFILE_VARIANT}" in  
    gpu)  
        profile_gpu  
        ;;  
    cpu)  
        profile_cpu  
        ;;  
    both)  
        profile_gpu  
        profile_cpu  
        ;;  
    *)  
        echo "Erro: PROFILE_VARIANT deve ser gpu, cpu ou both." >&2  
        exit 2  
        ;;  
esac  
  
echo  
echo "Perfis gerados em AULA-01-P-GPU/tarefa-20/resultados/perfis/"
```